

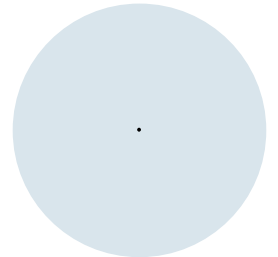
Einführung in die Programmierung (WIN+BIT)

Klassen I: Attribute, Konstruktoren, Instanz-Methoden und toString

Prof. Dr. Johannes C. Schneider / Prof. Dr. Markus Eiglsperger



Organisation von Source-Code: Klassen und Packages



Organisation von Source-Code

- Eine **Klasse** = eine **Datei** (Endung .java)
- Klassen **kapseln** eine gewisse Funktionalität
 - "abkapseln" von anderen Klassen
 - "bündeln"
- In den ersten Vorlesungen und Übungen haben wir das Compact Source Format (neu ab Java 21 verwendet)
 - Wir schreiben **eine** Klasse
 - diese hat eine **main-Methode** mit dem Code, den wir ausführen
 - Wir rufen hier auch Funktionen aus anderen Klassen des JDKs auf, z.B. Math oder Scanner
 - Wir definieren eigene Methoden in der Klasse

Klassen – Compact Source Code Format

```
void main() {
    boolean eingabe = true;
    Scanner scanner = new Scanner(System.in);
    while (eingabe) {
        System.out.print("Wochentag: ");
        int wochentag = scanner.nextInt();
        double entrittspreis = berechneEintrittspreis(wochentag);
        if (entrittspreis > 0) {
            System.out.printf("Eintrittspreis: %.2f\n", entrittspreis);
        } else {
            System.out.println("Ungültiger Wochentag");
        }
        System.out.print("Weiterer Eingabe J/N? ");
        String weiter = scanner.next();
        eingabe = weiter.toLowerCase().equals("j");
    }
}

double berechneEintrittspreis(int wochentag) {
    double result;
    switch (wochentag) {
        case 0 -> result = 4;
        case 1, 2 -> result = 5;
        case 3 -> result = 6;
        case 4 -> result = 4;
        case 5 -> result = 8;
        case 6 -> result = 10;
        default -> result = -1;
    }
    return result;
}
```

Compact Source Code Format - Limitationen

Um komplexe Programme übersichtlich zu halten muß der Code auf verschiedene Dateien verteilt werden, weil ansonsten die Datei zu groß und unübersichtlich wird.

Dies ist die Grundlage für objekt-orientiertes Design.

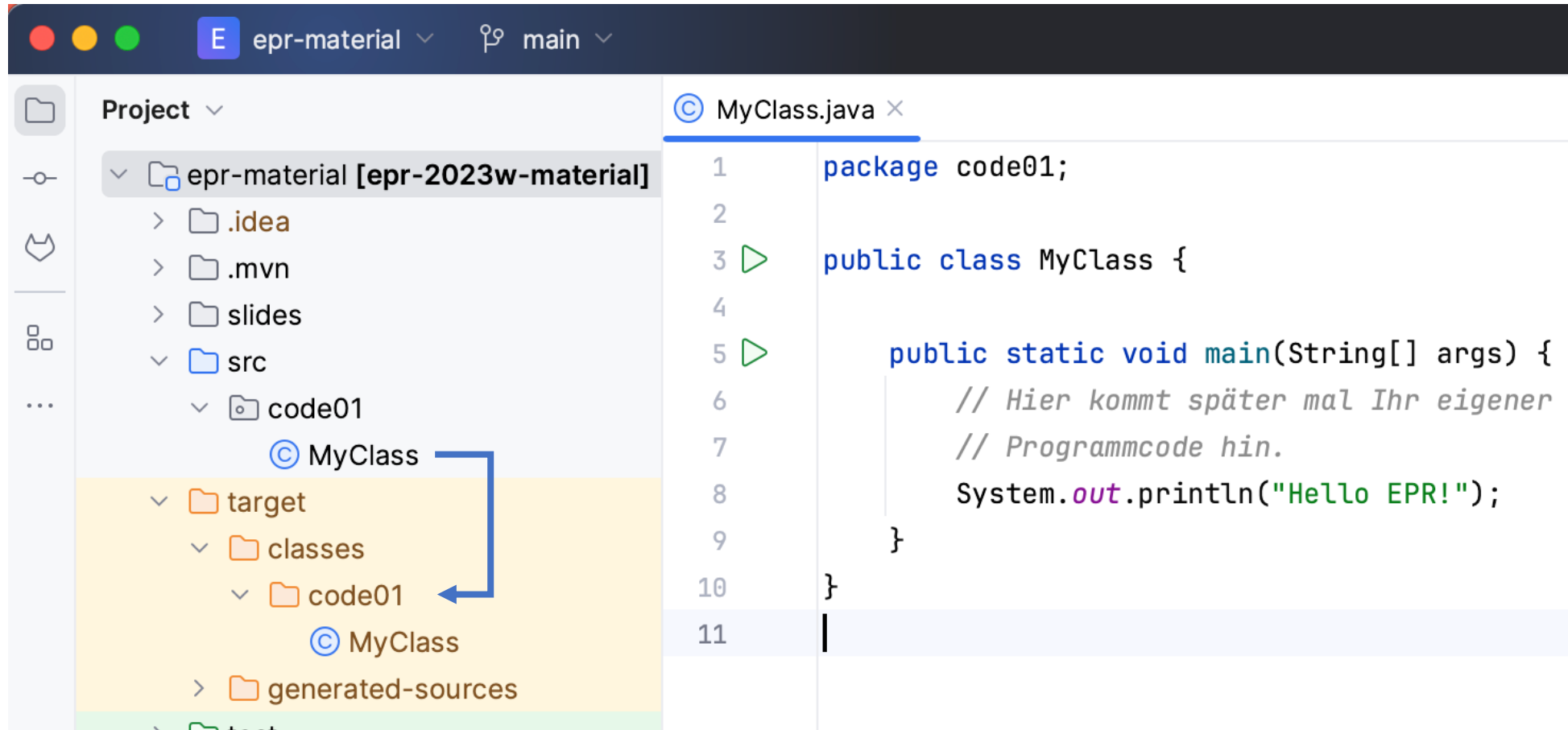
Aber:

- Programme im Compact Source Code Format können aber nicht auf verschiedene Dateien aufgeteilt werden.
- Es kann nur eine Datei mit dem gleichen Namen geben.

Lösung:

- Eine **Klasse** = eine **Datei** (Endung .java)
- Klassen **kapseln** eine gewisse Funktionalität
 - "abkapseln" von anderen Klassen
 - "bündeln»
- Klassen können Methoden anderer Klassen aufrufen, Daten speichern und zur Verfügung stellen.

Beispiel MyClass in IntelliJ



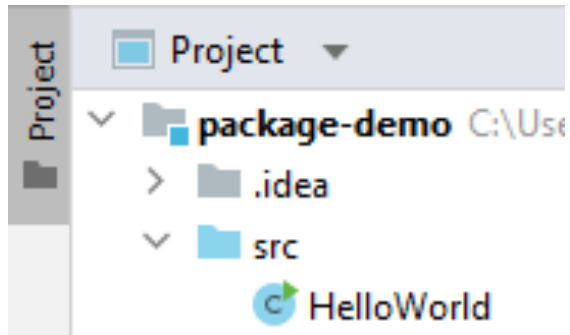
Klassen

```
package code01;  
  
public class MyClass {  
  
  
  
  
  
  
  
  
}
```

Packages

- Packages (Pakete) sind **Sammlungen von Klassen**
- Werden abgebildet **im Dateisystem als (Unter-)Verzeichnisse**
- Alles in src direkt ist im sogenannten **Default-Package** (ohne Namen)
- Alle weiteren Unterverzeichnisse entsprechen **benannten Packages**

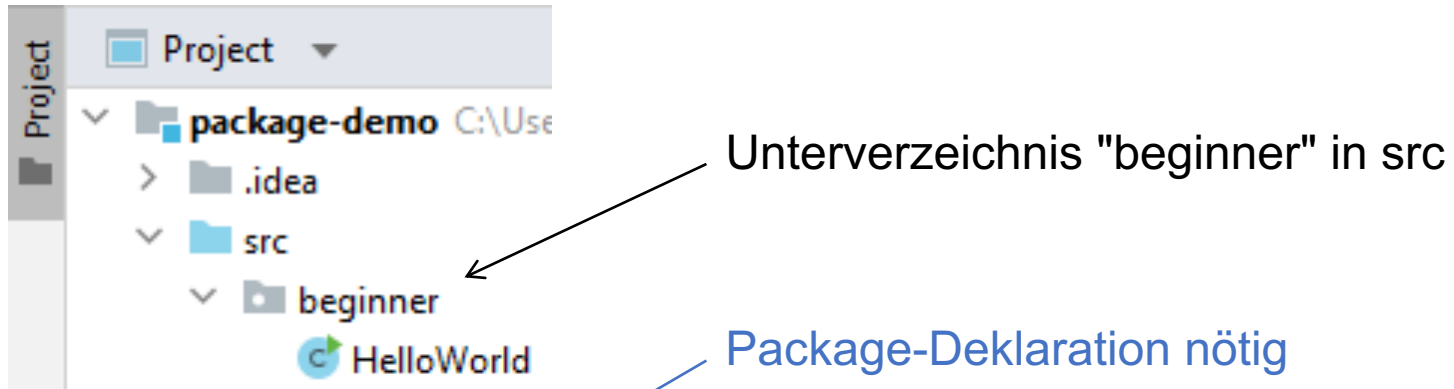
Default-Package



keine Package-Deklaration nötig

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

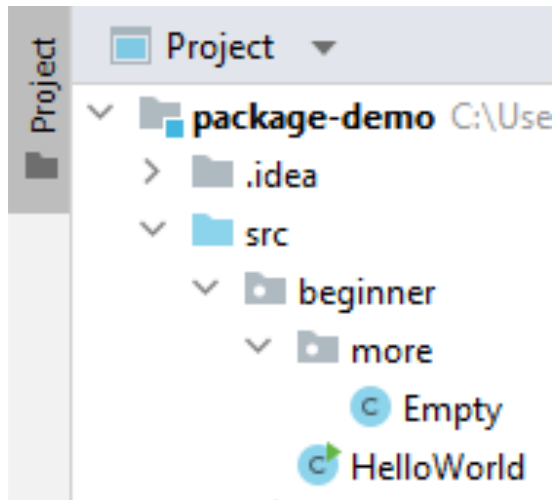
Benannte Packages



```
package beginner;

public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

Benanntes Package mit Unterpackage



- Package-Deklaration nötig
- Punkt als Trennzeichen für Unterverzeichnisse

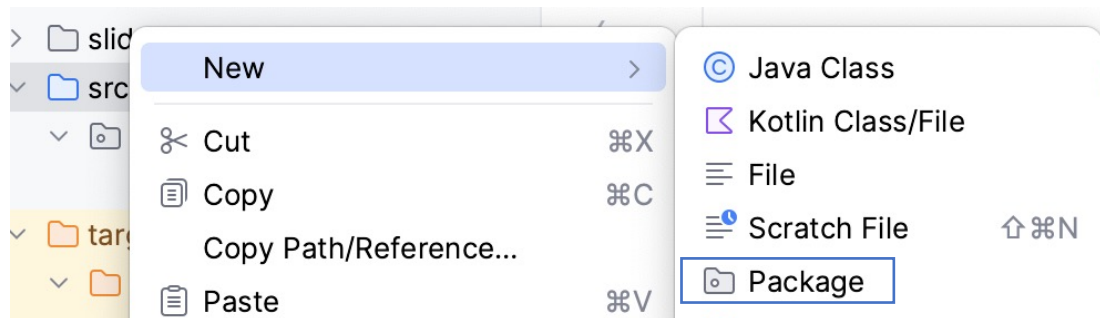
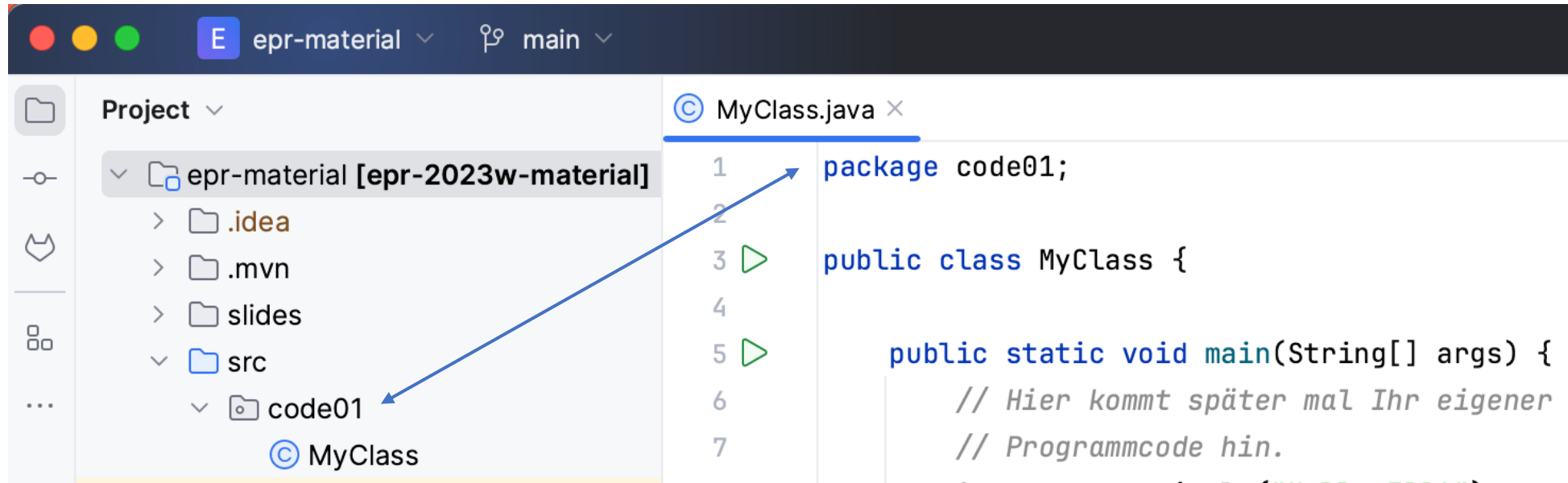
```
package beginner.more;
```

```
public class Empty {  
    // leere Klasse  
}
```

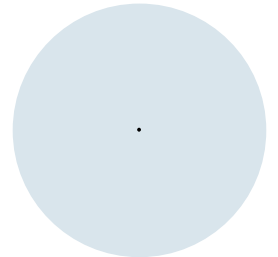
Paket-Namen

- Regeln für die Namen von Paketen: [Naming a package](#) (Java-Dokumentation)
 - TL;DR: nur Kleinbuchstaben
- Default-Package hat keinen Namen
- Java-Packages: `java.lang`, `java.util` und viele weitere
- Eigene Packages:
 - Niemals selbst `java.*` benutzen.
 - Sollte eindeutig sein (über mehrere Projekte hinweg)
 - Oft Domain-Bestandteile von hinten nach vorne, gefolgt von Projektname:
 - `org.springframework.security`
 - `com.github.codingdonkey.hexeditor.main`

Packages in IntelliJ



Verwendung von Klassen anderer Pakete



Vollqualifizierter Klassenname

- **1. Möglichkeit:** Referenzierung über vollqualifizierten Klassennamen

```
public class MyClass {  
    public static void main(String[] args) {  
        java.util.Scanner scanner =  
            new java.util.Scanner("Hallo!");  
    }  
}
```

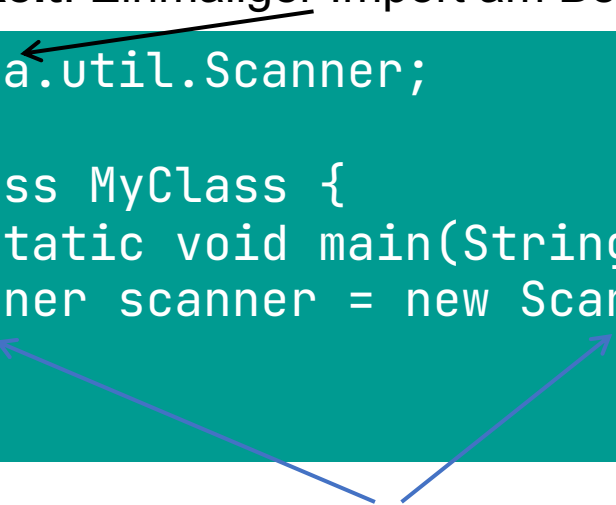


- an allen Stellen wird der vollqualifizierte Klassenname `java.util.Scanner` verwendet

import-Anweisung

- **2. Möglichkeit:** Einmaliger Import am Beginn der Klasse

```
import java.util.Scanner;  
  
public class MyClass {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner("Hallo!");  
    }  
}
```



- Dann Verwendung mit einfachem Klassennamen möglich

Ausnahme java.lang

Klassen im Paket `java.lang` können **ohne import** über einfachen Klassennamen verwendet werden.
Statt

```
public class MyClass {  
    public static void main(String[] args) {  
        java.lang.System.out.println(  
            java.lang.Math.PI);  
    }  
}
```

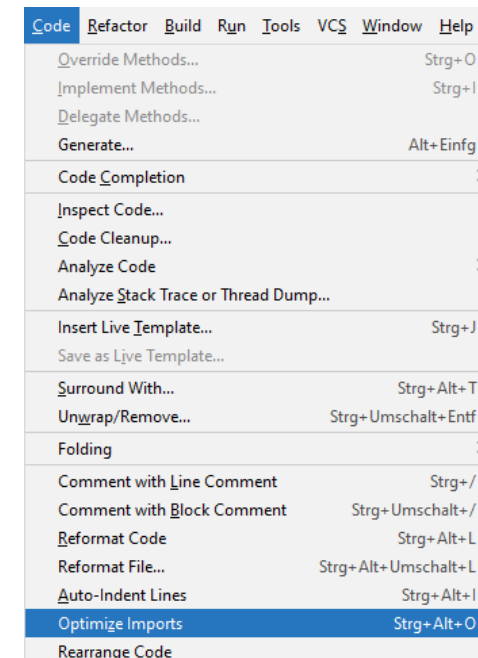
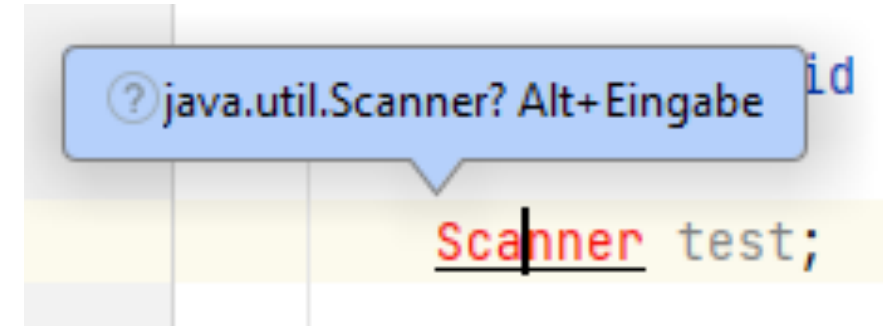
so nicht!

einfach

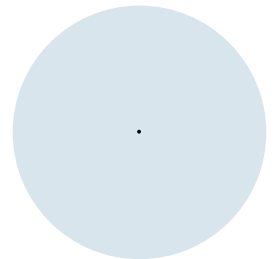
```
public class MyClass {  
    public static void main(String[] args) {  
        System.out.println(Math.PI);  
    }  
}
```

Unterstützung in IntelliJ

- Über Alt+Eingabe Klasse automatisch importieren
- Code → Optimize Imports entfernt nicht (mehr) gebrauchte import-Anweisungen



Klassen und Methoden, Übersicht



Klassendefinition

- Klassen kennen Sie schon: Ihr Code läuft in einer main-Methode einer Klasse, die Sie definiert haben:

```
public class Hello {  
    public static void main(String[] args) {  
        // Das ist ein Kommentar.  
        System.out.println("Hallo EPR!");  
    }  
}
```

Klassendefinition

Klassen-Methoden aufrufen

- Klassenmethoden (statische Methoden) haben wir schon behandelt:

```
double random = Math.random();
```

- Methoden-Aufrufe auf Klasseninstanzen haben Sie auch schon vorgenommen, z.B. bei Scanner:

```
Scanner inputScanner = new Scanner(System.in);  
String input = inputScanner.nextLine();
```

- Oder String:

```
String part = "Hallo".substring(1);
```

Klassen-Methoden definieren

Stück Programmcode, dem ein **Name** gegeben wird, unter dem es aufgerufen werden kann

Kann **Parameter** (Argumente) enthalten, die beim Aufruf durch Werte ersetzt werden

Kann ein Ergebnis **zurückliefern**

- Deklaration einer Methode

```
public static <Rückgabetyt> <Methodenname>(<FormaleParameter>) {  
    ...  
}
```

- <FormaleParameter> als komma-getrennte Liste "<Typ> <Name>, ..."

Klassen-Instanzen erzeugen

- Sie haben schon Instanzen von vorgegebenen Klassen mit dem Schlüsselwort `new` erzeugt:

```
Scanner inputScanner = new Scanner(System.in);
```

sog. "Konstruktor" einer Klasse

- Das geht bei Scanner auf mehrere Arten:

```
Scanner stringScanner = new Scanner(myString);  
Scanner fileScanner = new Scanner(path, "UTF-8");
```

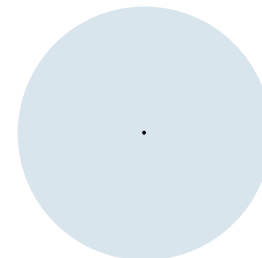
- Sie können Scanner also für `System.in`, für Strings und für Dateien instanziiieren.

Inhalt der folgenden Kapitel

In den folgenden Kapiteln lernen Sie, wie Sie eigene Klassen definieren können mit

- Attributen (Werte, die innerhalb einer Klasse gespeichert werden)
- Konstruktoren (Mechanismus zum Erzeugen von Instanzen)
- Methoden (Funktionen mit Parametern und Rückgabewert)
- ...

Klassen und Attribute



Motivation für den Einsatz von Klassen

- Aufgabe: Programm zur Adressverwaltung (20 Adressen)
- Adress-Eintrag besteht aus:
 - Name
 - Straße
 - Wohnort
 - Postleitzahl

Adressverwaltung: Naiver Ansatz

```
final int MAX_ADDRESSES = 20;
String[] name      = new String[MAX_ADDRESSES];
String[] street    = new String[MAX_ADDRESSES];
String[] city      = new String[MAX_ADDRESSES];
int[]    postcode  = new int[MAX_ADDRESSES];

name[5] = "Chris"; street[5] = "Lindenstr."; city[5] = "Konstanz";
postcode[5] = 78462;
```

- Nachteile:
- Müssen immer 4 Arrays "herumreichen"
- Müssen dafür sorgen, dass die Arrays "synchron" bleiben
- Neues Feld (z.B. favorit, email) erfordert sehr viel Änderungsaufwand



code12.b_primitive
Address

Adressverwaltung: Verwendung von Klassen

- Zusammenfassung aller zu einer Adresse gehörenden Daten wäre schön.
- Prinzip: "Klasse" (eigener Datentyp)
 - Zusammenfassung verschiedener Variablen
 - Zusammenfassung von
 - **Daten** und
 - **Operationen** auf den Daten

Vorschau:

```
final int MAX_ADDRESSES = 20;
public class Address {
    public String name;
    public String street;
    public String city;
    public int postcode;
    public boolean favorite;
}

Address[] addresses = new Address[MAX_ADDRESSES];
for (int i = 0; i < addresses.length; i++) {
    addresses[i] = new Address();
}

addresses[5].name = "Chris";
addresses[5].city = "Konstanz";
addresses[5].postcode = 78462; // usw.
```

Definition einer Klasse

```
public class <Klassenname> {
    <Deklarationen>
}
```

Mögliche <Deklarationen> sind:

- Attribute: public <Datentyp> <Attributname>;
- Methoden*: public <Datentyp> <Methodenname>(<FormaleParameter>) {
 ...
 }
- Konstruktoren*: public <Klassenname>() { ... }

*behandeln wir später...

Klasse Address

Achtung, Fehlergefahr: Auf deutsch Adresse,
auf Englisch Address

```
public class Address {  
    public String name;  
    public String street;  
    public String city;  
    public int postcode;  
    public boolean favorite;  
}
```

- name, street, city, postcode und favorite sind Attribute der Klasse Address
- public bedeutet, dass man von überall auf sie zugreifen kann
 - Attribute sind eigentlich interne Details und sollten geschützt werden
 - mehr zu (eingeschränkter) Sichtbarkeit später, zunächst alles public deklariert
- Address besteht nur aus der Definition von Attributen.
 - Methoden und Konstruktoren folgen später...



code12.c_basic
Address

Instanziierung

- Objekterzeugung

- `<Klassenname> <Instanzname>;`

- `<Instanzname> = new <Klassenname>();`

```
Address address;
```

```
address = new Address();
```

- Bzw. zusammengefasst

- `<Klassenname> <Instanzname> = new <Klassenname>();`

```
Address address = new Address();
```

Schon einmal merken für später: Das ist der sog. "Konstruktor" einer Klasse. Jede Klasse erhält automatisch einen parameterlosen Konstruktor der Form `Klassenname()`



code12.c_basic
Main1

Attribut-Zugriff

- <Instanzname>.<Attributname>
- Beispiel

```
Address address = new Address();  
address.name = "Chris";  
address.street = "Lindenstr.";  
address.city = "Konstanz";  
address.postcode = 78462;  
  
System.out.println(address.city);  
int nextPostCode = address.postcode + 1;
```



code12.c_basic
Main1

Eigene Klasse als Methoden-Parameter (1)

- Praktisch: Alle Informationen in einem Parameter address (anstatt 4 einzelne Parameter name, street, postcode, city und favorite):

```
public static void printAddress(Address address) {  
    System.out.printf("%s wohnt in %s, %d %s",  
        address.name, address.street,  
        address.postcode, address.city);  
}
```



code12.c_basic
Main2

Eigene Klasse als Methoden-Rückgabewert

- Auch als Rückgabetyt möglich:

```
private static Address createRandomAddress() {  
    Address address = new Address();  
    address.name = Math.random() < 0.5 ? "Max" : "Beate";  
    address.street = Math.random() < 0.5 ? "Hauptstraße " : "Hintergasse ";  
    address.street += (int) (Math.random() * 20) + 1;  
    address.city = Math.random() < 0.5 ? "Neustadt" : "Zufallsdorf";  
    address.postcode = (int) (Math.random() * 100000);  
    address.favorite = Math.random() < 0.5;  
    return address;  
}
```



code12.c_basic
Main3

Adressverwaltung: Objekt-Orientierter Ansatz

- Mit diesem Wissen können wir für unsere Adressverwaltung jetzt die Klasse Address verwenden:

```
public class Address {  
    public String name;  
    public String street;  
    public String city;  
    public int postcode;  
    public boolean favorite;  
}
```

- ... und ein Array von Adress-Instanzen bilden (man kann auch Arrays über eigene Datentypen bilden):

```
final int MAX_ADDRESSES = 20;  
Address[] addresses = new Address[MAX_ADDRESSES];
```



code12.c_basic
AddressBook

Adressverwaltung: 3. Objekt-Orientierter Ansatz

```
final int MAX_ADDRESSES = 20;
public class Address {
    public String name;
    public String street;
    public String city;
    public int postcode;
    public boolean favorite;
}
Address[] addresses = new Address[MAX_ADDRESSES];
for (int i = 0; i < addresses.length; i++) {
    addresses[i] = new Address();
}
```

Array-Initialisierung mit "leerer"
Adresse



code12.c_basic
AddressBook

Adressverwaltung: Objekt-Orientierter Ansatz, Vorteil

```
final int MAX_ADDRESSES = 20;
public class Address {
    public String name;
    public String street;
    public String city;
    public int postcode;
    public String email;
    public boolean favorite;
}
Address[] addresses = new Address[MAX_ADDRESSES];
for (int i = 0; i < addresses.length; i++) {
    addresses[i] = new Address();
}
```

Erweiterung um weiteres Attribute
email problemlos möglich